

R Scripts

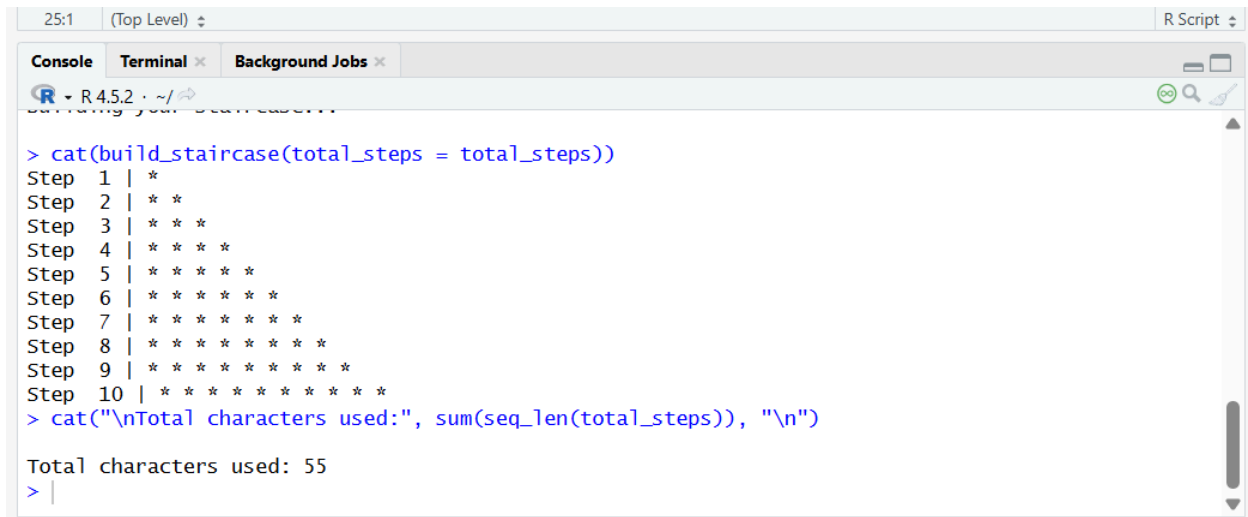
ITP321 Finals Project

Members:

Francis, John Carlo
Nacionales, Maranello
Orlanda, Ashlie Mae

SECTION 1 FOR LOOP

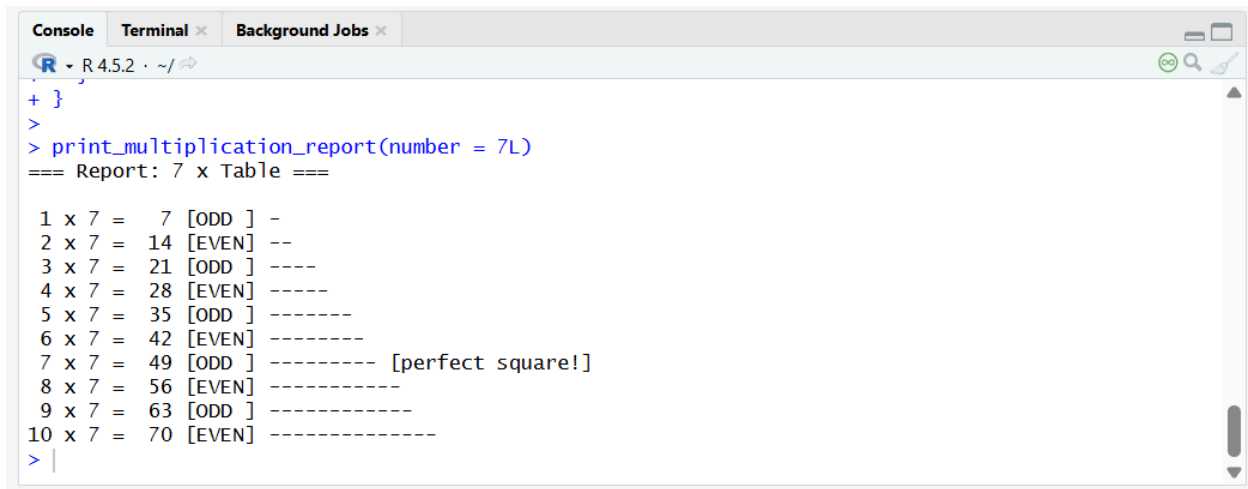
For Loop 1



```
25:1 (Top Level) R Script
Console Terminal Background Jobs
> cat(build_staircase(total_steps = total_steps))
Step 1 | *
Step 2 | * *
Step 3 | * * *
Step 4 | * * * *
Step 5 | * * * * *
Step 6 | * * * * * *
Step 7 | * * * * * * *
Step 8 | * * * * * * * *
Step 9 | * * * * * * * * *
Step 10 | * * * * * * * * * *
> cat("\nTotal characters used:", sum(seq_len(total_steps)), "\n")
Total characters used: 55
>
```

This code defines a function called `build_staircase` that generates a visual staircase pattern using a for loop. It begins by validating the input to ensure that the number of steps is a valid positive number, preventing errors or unexpected behavior. Inside the loop, the function builds each step by repeating a chosen character and formatting it into a readable structure. The results are accumulated into a single string, which is then printed all at once for better readability. The script also calculates the total number of characters used, demonstrating how loops can be used not only for repetition but also for building structured and formatted output.

For Loop 2

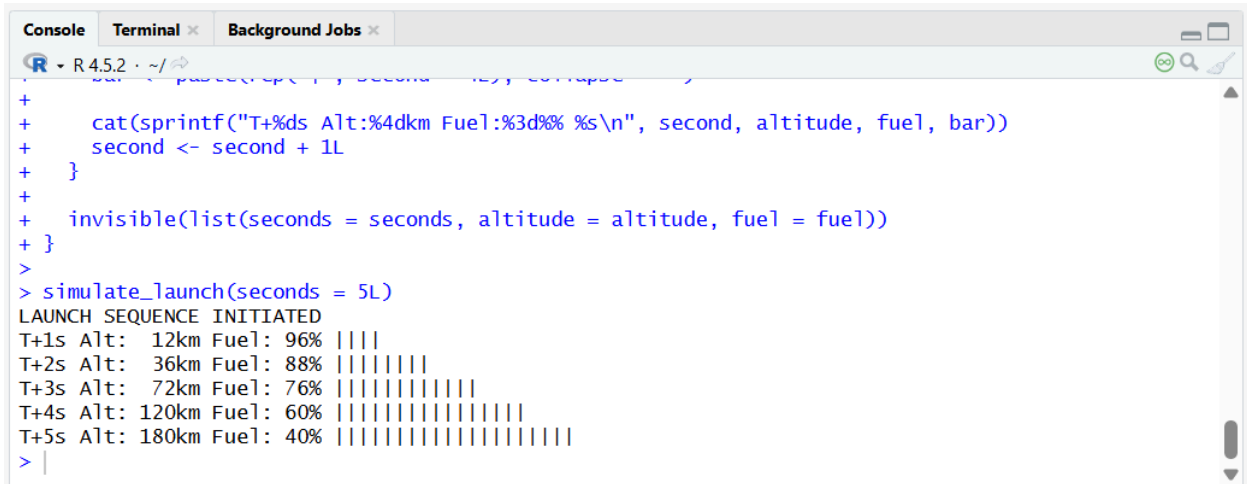


```
Console Terminal Background Jobs
R 4.5.2 ~/  
+ }  
>  
> print_multiplication_report(number = 7L)  
=== Report: 7 x Table ===  
  
1 x 7 = 7 [ODD ] -  
2 x 7 = 14 [EVEN] --  
3 x 7 = 21 [ODD ] ----  
4 x 7 = 28 [EVEN] -----  
5 x 7 = 35 [ODD ] -----  
6 x 7 = 42 [EVEN] -----  
7 x 7 = 49 [ODD ] ----- [perfect square!]  
8 x 7 = 56 [EVEN] -----  
9 x 7 = 63 [ODD ] -----  
10 x 7 = 70 [EVEN] -----  
> |
```

This code demonstrates a more advanced use of a for loop by generating a detailed multiplication table report. It includes a helper function, `is_perfect_square`, which checks whether a number is a perfect square. The main function, `print_multiplication_report`, iterates through a sequence of multipliers, calculates each product, and adds additional interpretations such as whether the result is even or odd and whether it is a perfect square. It also includes a simple visual bar to represent the size of the number. This example shows how loops can be used to create informative, formatted reports rather than just simple numeric output.

SECTION 2 — WHILE LOOP

While Loop 1



```
Console Terminal Background Jobs
R 4.5.2 ~ /
+   cat(sprintf("T+%ds Alt:%4dkm Fuel:%3d%% %s\n", second, altitude, fuel, bar))
+   second <- second + 1L
+ }
+   invisible(list(seconds = seconds, altitude = altitude, fuel = fuel))
+ }
>
> simulate_launch(seconds = 5L)
LAUNCH SEQUENCE INITIATED
T+1s Alt: 12km Fuel: 96% ||||
T+2s Alt: 36km Fuel: 88% |||||||
T+3s Alt: 72km Fuel: 76% |||||||||
T+4s Alt: 120km Fuel: 60% |||||||||
T+5s Alt: 180km Fuel: 40% |||||||||
> |
```

This code defines a function named `simulate_launch` that uses a while loop to simulate a simple launch sequence over a number of seconds. The loop continues running as long as the current time step is within the specified duration, making it suitable for condition-based iteration. During each iteration, the function updates key variables such as altitude and fuel to represent changes over time, mimicking a real-world simulation. It also generates a visual progress indicator using repeated characters to make the output easier to follow. The use of formatted output provides a structured display of the simulation at each step. This example demonstrates how while loops are useful for modeling dynamic systems where values evolve continuously based on a condition.

While Loop 2

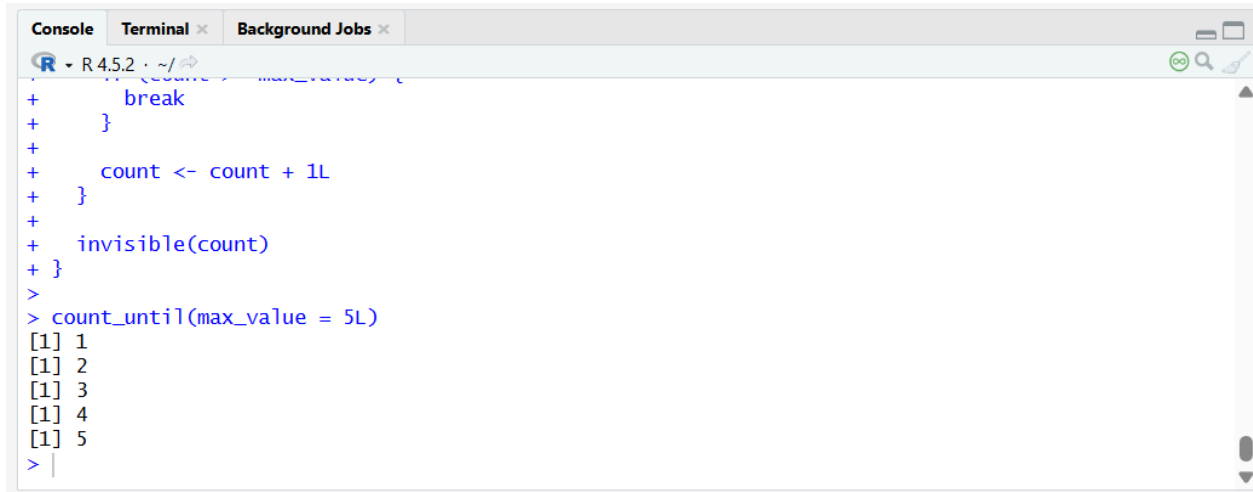


```
R R 4.5.2 ~/  
+   label  
+   ))  
+   second <- second - 1L  
+ }  
+ invisible(NULL)  
+ }  
>  
> run_auction_countdown()  
[5] $500 !!!!! Plenty of time...  
[4] $485 !!!!! Place your bids!  
[3] $470 !!! Going once...  
[2] $455 !! Going twice...  
[1] $440 ! FINAL CHANCE!  
> |
```

This code implements a function called `run_auction_countdown` that simulates an auction countdown using a while loop. The loop runs as long as there is remaining time, decrementing the countdown value at each step. A lookup table is used to assign descriptive messages based on the remaining time, which simplifies the logic compared to multiple conditional statements. At each iteration, the function calculates a changing value (such as a bid amount) to simulate progression over time and prints a formatted message that includes urgency indicators. This example highlights how while loops can effectively model countdown processes and time-based events, while also demonstrating clean coding practices through the use of structured data for messaging.

SECTION 3

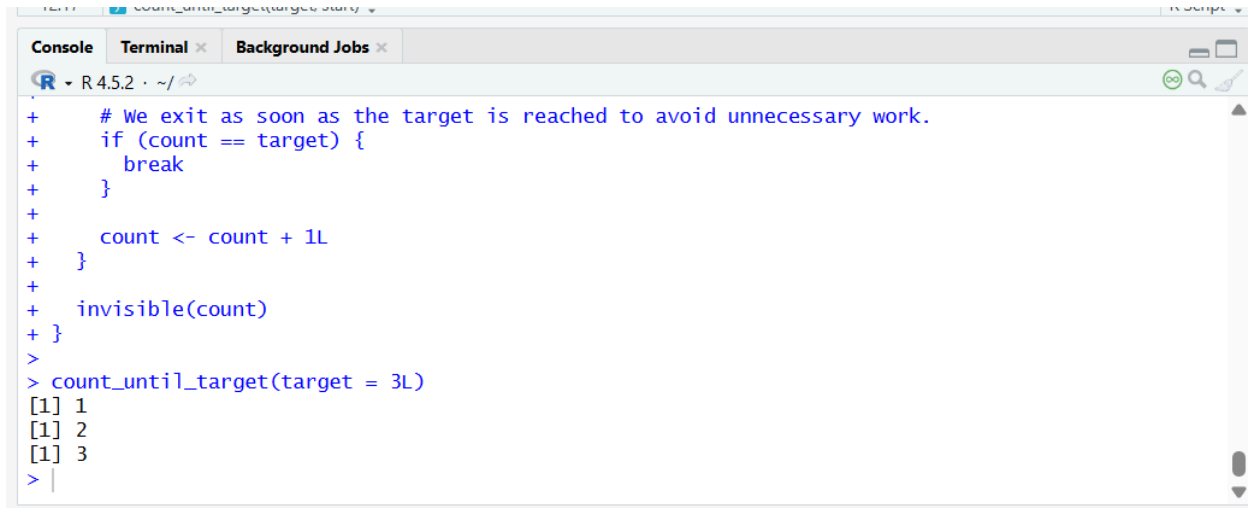
REPEAT LOOP 1



```
Console Terminal Background Jobs
R 4.5.2 ~ /
+ count_until(max_value) {
+   break
+ }
+   count <- count + 1L
+ }
+ invisible(count)
+ }
>
> count_until(max_value = 5L)
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
> |
```

This code demonstrates the use of a repeat loop through a function called `count_until`. Unlike other loops, a repeat loop runs indefinitely unless explicitly stopped. In this case, the loop prints a counter value and checks whether it has reached a specified maximum value. Once the condition is met, the `break` statement terminates the loop. This example illustrates how repeat loops are useful when the stopping condition needs to be controlled internally, such as in iterative processes or user-driven tasks.

REPEAT LOOP 2

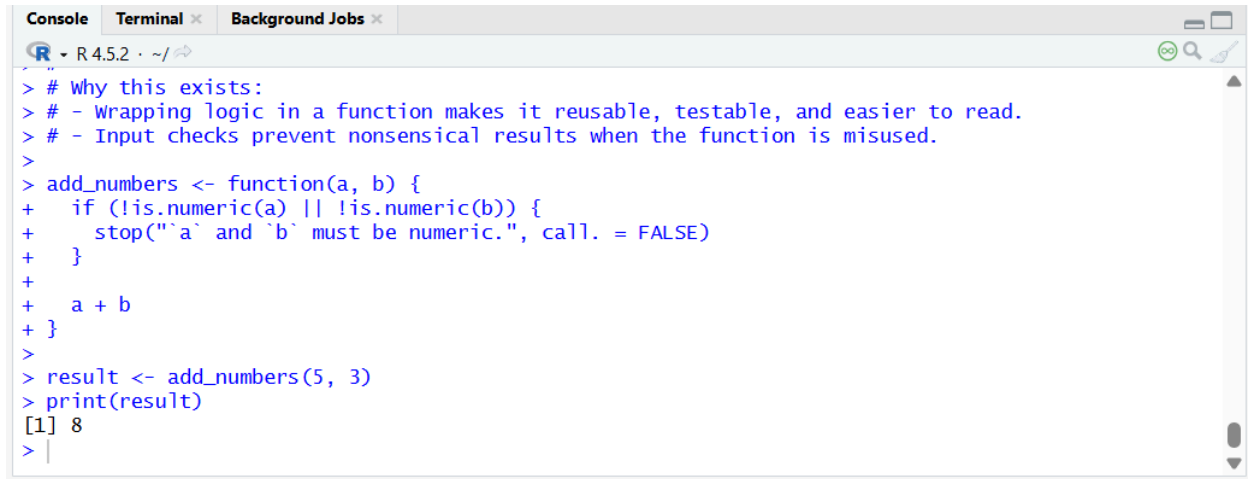


```
R 4.5.2 ~/  
+ # We exit as soon as the target is reached to avoid unnecessary work.  
+ if (count == target) {  
+   break  
+ }  
+  
+   count <- count + 1L  
+ }  
+ invisible(count)  
+ }  
>  
> count_until_target(target = 3L)  
[1] 1  
[1] 2  
[1] 3  
> |
```

This code defines a function called `count_until_target` that demonstrates the use of a repeat loop, which runs indefinitely until explicitly stopped. The function initializes a counter starting from a given value and continuously prints it at each iteration. Instead of looping for a fixed number of times, the loop checks whether the current value has reached a specified target, and immediately exits using a `break` statement once the condition is met. This approach avoids unnecessary iterations and reflects real-world scenarios where a process stops as soon as a goal is achieved, such as searching for a specific value. The function returns the final count invisibly, emphasizing efficiency and control in loop execution.

SECTION 4

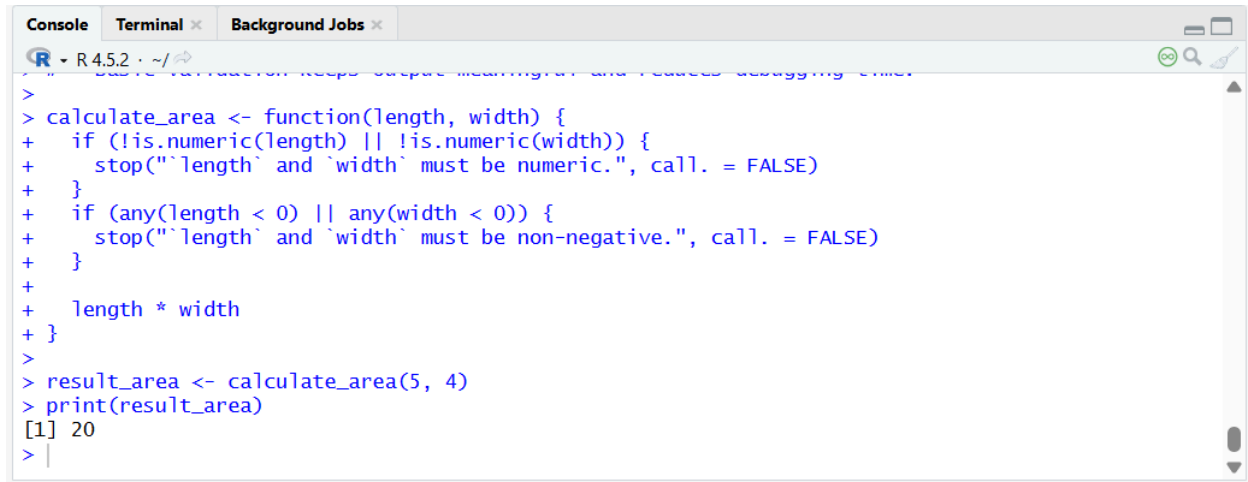
FUNCTION 1



```
Console Terminal Background Jobs
R - R 4.5.2 - ~/
> # Why this exists:
> # - Wrapping logic in a function makes it reusable, testable, and easier to read.
> # - Input checks prevent nonsensical results when the function is misused.
>
> add_numbers <- function(a, b) {
+   if (!is.numeric(a) || !is.numeric(b)) {
+     stop("`a` and `b` must be numeric.", call. = FALSE)
+   }
+   a + b
+ }
>
> result <- add_numbers(5, 3)
> print(result)
[1] 8
> |
```

This code defines a simple function called `add_numbers` that takes two inputs and returns their sum. Before performing the calculation, the function checks whether both inputs are numeric, ensuring that the operation is valid and preventing errors. By wrapping this logic inside a function, the code becomes reusable, easier to read, and safer to use in different contexts. The function is then called with example values, and the result is printed.

Function 2

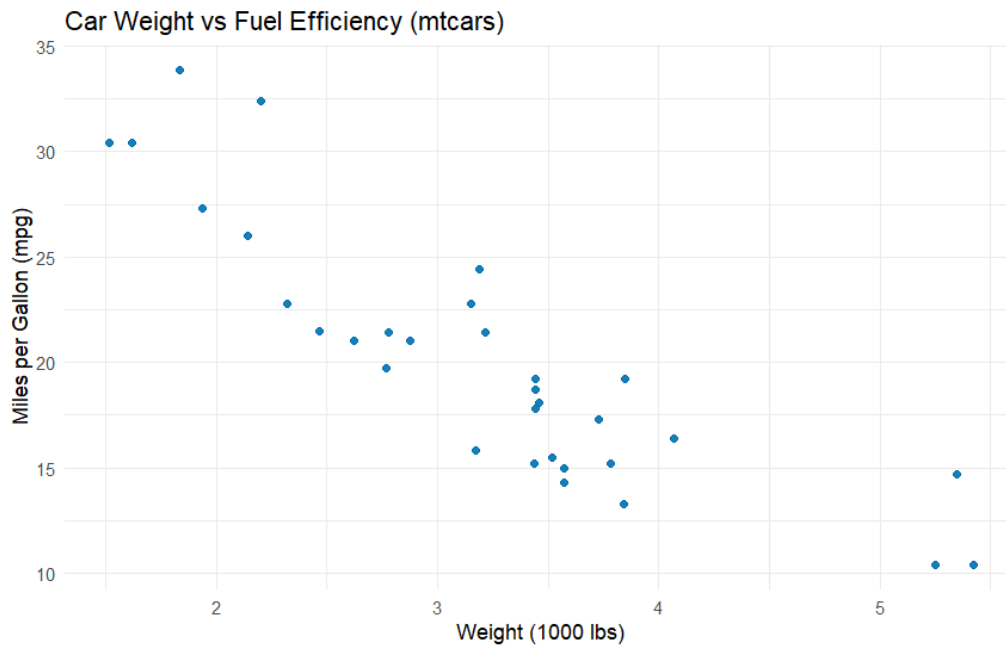


```
Console Terminal Background Jobs
R 4.5.2 ~/  
> calculate_area <- function(length, width) {  
+   if (!is.numeric(length) || !is.numeric(width)) {  
+     stop("`length` and `width` must be numeric.", call. = FALSE)  
+   }  
+   if (any(length < 0) || any(width < 0)) {  
+     stop("`length` and `width` must be non-negative.", call. = FALSE)  
+   }  
+   length * width  
+ }  
>  
> result_area <- calculate_area(5, 4)  
> print(result_area)  
[1] 20  
> |
```

This code defines a function named `calculate_area` that computes the area of a rectangle by multiplying its length and width. It includes input validation to ensure that both values are numeric and non-negative, which helps maintain meaningful results and avoids incorrect calculations. The function is then executed with sample values, demonstrating how functions can encapsulate logic and enforce rules for proper usage.

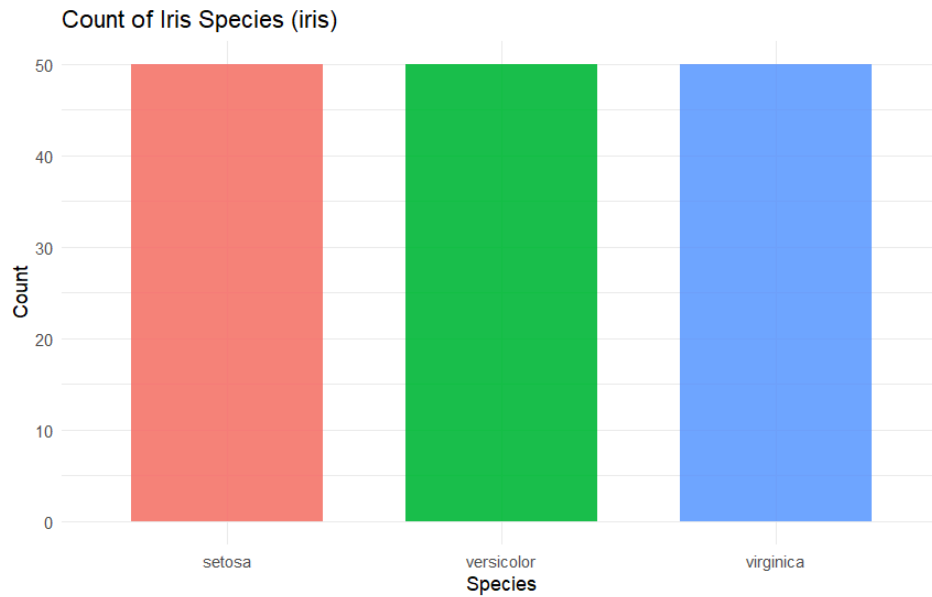
SECTION 5 — DATA VISUALIZATION

Data Visualization 1:



This code creates a bar chart using the iris dataset to display the number of observations for each species. It first ensures that the ggplot2 package is available, then maps the species variable to the x-axis and uses `geom_bar()` to automatically count the occurrences of each category. The plot is labeled clearly with a title and axis labels, and the legend is removed since it is redundant. This visualization provides a quick way to check the distribution of categories in the dataset before performing deeper analysis.

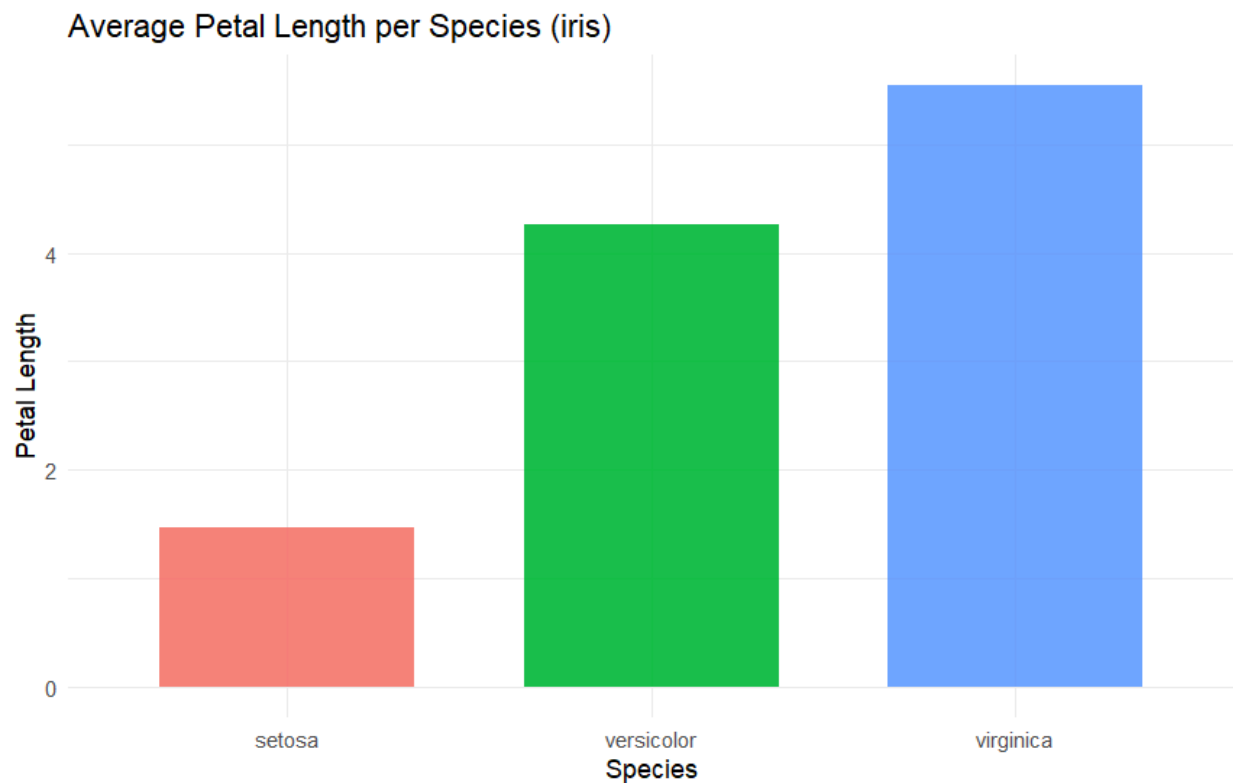
Data Visualization 2:



This code generates a scatter plot using the mtcars dataset to examine the relationship between car weight and fuel efficiency. It maps weight to the x-axis and miles per gallon to the y-axis, then plots each observation as a point. The use of a clean theme improves readability and presentation. This type of visualization is useful for identifying trends, correlations, and potential outliers in numerical data.

This code begins by setting up an automatic package-checking system to ensure that the required library (ggplot2) is installed, making the script reproducible and easy to run on any system. It then loads the built-in mtcars dataset, which contains various attributes of cars, ensuring that no external files are needed. A function named fit_mpg_vs_weight is defined to create a linear regression model using the lm() function, where fuel efficiency (mpg) is predicted based on car weight (wt). Another function, plot_mpg_vs_weight, generates a scatter plot with a fitted regression line, allowing the user to visually assess the relationship between the variables. Finally, the script runs the analysis by fitting the model, printing its statistical summary, and displaying the plot, combining both numerical and visual insights to better understand the data.

Data Science 2



This code ensures that the required packages (dplyr and ggplot2) are available before proceeding, promoting portability and smooth execution. It uses the built-in iris dataset, which contains measurements of different flower species. A function called `summarize_petal_length` groups the data by species and calculates both the number of observations and the average petal length for each group, providing a clear statistical comparison. Another function, `plot_avg_petal_length`, visualizes these averages using a bar chart, making it easier to compare the species visually. The script concludes by printing both the summary table and the corresponding plot, effectively combining numerical summaries with graphical representation for clearer interpretation.

